Assignment 1: Please complete the given instruction below

Submission:

1) Please submit the program/solutions, and the screenshots of the output.

2) Please submit a recorded-video which explains how you implement the assignment and demonstrate how it works. This will give everyone the opportunity to present their Assignment-solution. Please record a video (2~10 minutes) with the following contents:

- Introduce yourself *(I would like to see your face in some part of the video, so I can recognize you later when we see each other at the campus :))*
- Show/demonstrate how your assignment works.
- Explain your code (walkthrough), how you design it (a quick/detailed walk-through of the commands/scripts/programming)
- Speak about challenges that you faced during this Assignment.
- Evaluate your self. Have you implemented all requirements of the assignment? how do you evaluate yourself out of 10 for this assignment?

NOTE: You can record the video using some screen-capture software (like OBS : https://obsproject.com/). To submit the video:

1. You can upload the video on the youtube (you may make it unlisted) and submit the link here.
2. You can also directly upload the video to the BB.
3. If you are running out of time, You can submit the assignment by deadline and submit the video within 24 hours after the deadline (as 2nd attempt)
4. Without Video recording you can only earn a maximum of 1/3 of the assignment mark.

Assignment 1: Parallel Binary File Processing with Resource Monitoring

In this assignment, you will design and implement a **mini operating-system style monitoring system** that combines user-space processes, inter-process communication, file I/O, logging, and kernel interaction.

You are not just writing a program—you are building a **multi-process system** that behaves like a real resource-monitoring service:

- Worker processes read large binary files in parallel
- Each worker actively monitors its own memory usage
- When memory usage becomes dangerous, the worker immediately notifies the parent
- The parent coordinates all workers and records system events
- All activity is safely logged using file locking
- A kernel module is contacted using ioctl() for system-wide monitoring

This assignment simulates how **real operating systems, servers, and security tools** track resource usage, detect abnormal behavior, and coordinate multiple processes safely.

By the end of this assignment, you will have practical experience with:

- Process creation and parallelism using fork()
- Streaming large files using blocking and non-blocking I/O
- Monitoring memory through the Linux /proc filesystem
- Signaling between processes in real time
- Safe concurrent logging using file locks
- Communicating with kernel space using ioctl()

This is not a “toy” program—every technique used here exists in production systems such as web servers, intrusion detection tools, and performance monitors.

Consider Naming Convention

To ensure uniqueness, all function names must include your initials.

For example, if your initials are AB, name your functions like:

- void ab_generate_binary_file()
- void ab_worker_process()
- void ab_log_event()

Replace "XX" with your initials throughout your code.

Part 1: Dynamic File Generation

In this step, you will generate test binary files dynamically (using below template) instead of using predefined data files.

Task Breakdown

Each worker process will process a binary file. Instead of manually creating these files, students will:

✅ Prompt the user for file sizes (in MB)

✅ Create binary files filled with random data

✅ Ensure different file sizes per execution

Modify the function below to:

✅ Ask the user for file sizes

✅ Generate binary files dynamically

Complete This Function

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>

void xx_generate_binary_file(const char *filename, int size_mb) {
    int fd = open(filename, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) {
        perror("File creation failed");
        exit(1);
    }

    char buffer[1024 * 1024]; // 1MB buffer
    memset(buffer, 'X', sizeof(buffer)); // Fill with dummy data

    for (int i = 0; i < size_mb; i++) {
        write(fd, buffer, sizeof(buffer));
    }

    close(fd);
```

```
}

int main() {
    int size1, size2, size3;
    printf("Enter file size for Worker 1 (MB): ");
    scanf("%d", &size1);
    printf("Enter file size for Worker 2 (MB): ");
    scanf("%d", &size2);
    printf("Enter file size for Worker 3 (MB): ");
    scanf("%d", &size3);

    xx_generate_binary_file("worker1.bin", size1);
    xx_generate_binary_file("worker2.bin", size2);
    xx_generate_binary_file("worker3.bin", size3);

    printf("Binary files created.\n");
    return 0;
}
```

💡 Conceptual Questions:

1. Why do we use O_CREAT | O_TRUNC in open()?
2. What happens if write() fails inside the loop?
3. Why do we use memset() before writing to the file?

Part 2: Worker Processes

In this step, you will use the below code to reading binary files in parallel, read in chunks instead of reading the whole file, Dynamically tracks memory usage in real-time, Notifies the parent immediately when memory usage is high or when processing is complete.

Task Breakdown

Each worker process should:

1. Open a binary file in read-only mode.
2. Read 4KB at a time (to simulate streaming or large file processing).
3. Monitor its own memory usage via /proc/self/status.

4. If memory usage exceeds 50MB, notify the parent process using SIGUSR1.
5. When the file is fully read, notify the parent process using SIGUSR2.

Modify the function below to:

✅ Read 4KB chunks from the binary file

✅ Check memory usage using /proc/[pid]/status

✅ Send SIGUSR1 if memory > 50MB

✅ Send SIGUSR2 when the file is fully read

✍️ Complete This Function

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <signal.h>

void xx_worker_process(const char *filename, pid_t parent_pid) {
    int fd = open(filename, O_RDONLY);
    if (fd < 0) {
        perror("Worker: Failed to open file");
        exit(1);
    }

    char buffer[4096]; // 4KB buffer
    ssize_t bytes_read;
    int mem_usage = 0;

    while ((bytes_read = read(fd, buffer, sizeof(buffer))) > 0) {
        FILE *status_file = fopen("/proc/self/status", "r");
        if (!status_file) {
            perror("Failed to open /proc/self/status");
            exit(1);
        }

        char line[256];
        while (fgets(line, sizeof(line), status_file)) {
            if (strncmp(line, "VmRSS:", 6) == 0) {
                sscanf(line + 6, "%d", &mem_usage);
```

```
                    break;
                }
            }
            fclose(status_file);

            if (mem_usage > 50000) {
                kill(parent_pid, SIGUSR1);
            }
        }

        close(fd);
        printf("Worker completed: %s\n", filename);
        kill(parent_pid, SIGUSR2);
        exit(0);
    }
```

💡 Conceptual Questions:

1. Why do we check /proc/self/status for memory usage?
2. What would happen if kill(parent_pid, SIGUSR1); fails?
3. How does read() behave differently in blocking vs non-blocking mode?

NOTE: If we only read the binary file in 4KB chunks without storing the data, the memory usage will stay very low because the kernel efficiently manages the read buffer. If VmPeak never exceeds 2100KB, it will never reach the 50MB threshold, meaning SIGUSR1 is never sent.

To actually trigger SIGUSR1, we need to increase memory consumption. Here are two ideas:

1) Instead of just reading and discarding data, accumulate it in an array or linked list. In this way, Memory will grow, eventually exceeding 50MB, triggering SIGUSR1.

2) If we want constant high memory usage, allocate a large array at the start.

Part 3: Parent Process

You are asked to use below template to handle worker signals with sigaction(), Log high memory usage when SIGUSR1 is received from a worker, Log worker completion when

SIGUSR2 is received from a worker.

Task Breakdown

1. Worker processes send SIGUSR1 if memory usage exceeds 50MB.
2. Worker processes send SIGUSR2 when they complete processing their file.
3. The parent process must use sigaction() to properly handle both signals.
4. The parent should log which worker sent the signal (using siginfo_t).

Modify the signal handlers to:

✅ SIGUSR1: Logs high memory usage

✅ SIGUSR2: Logs worker completion

✍️ Complete This Function

```
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>

void xx_signal_handler(int sig) {
    if (sig == SIGUSR1) {
        printf("Worker process exceeded memory limit!\n");
    } else if (sig == SIGUSR2) {
        printf("Worker process completed its task.\n");
    }
}

void xx_setup_signals() {
    signal(SIGUSR1, xx_signal_handler);
    signal(SIGUSR2, xx_signal_handler);
}
```

💡 Conceptual Questions:

1. What happens if two workers send SIGUSR1 at the same time?
2. Why is signal(SIGUSR1, xx_signal_handler); needed in main()?

Part 4: Secure Logging

You are asked to use the below template to Write logs to syslog.log to record worker memory issues and completion, Use fcntl() to apply file locking before writing, preventing simultaneous writes from multiple workers, and lock the file-write to prevent corruption using fnctl().

Task Breakdown

1. When a worker sends SIGUSR1, the parent logs a memory warning.
2. When a worker sends SIGUSR2, the parent logs a worker completion message.
3. To prevent corruption, file writes must be locked using fcntl().
4. Once a log entry is written, the lock should be released.

Modify xx_log_event() to:

✅ Write logs to syslog.log

✅ Use fcntl() to lock the file before writing

✍️ Complete This Function

```
#include <fcntl.h>
#include <unistd.h>

void xx_log_event(const char *message) {
    int fd = open("syslog.log", O_WRONLY | O_APPEND | O_CREAT, 0644);
    if (fd < 0) {
        perror("Log file open failed");
        return;
    }

    struct flock lock;
    lock.l_type = F_WRLCK;
    lock.l_whence = SEEK_SET;
    lock.l_start = 0;
    lock.l_len = 0;
    lock.l_pid = getpid();

    if (fcntl(fd, F_SETLK, &lock) == -1) {
```

```
            perror("File lock failed");
            close(fd);
            return;
        }

        write(fd, message, strlen(message));

        lock.l_type = F_UNLCK;
        fcntl(fd, F_SETLK, &lock);

        close(fd);
    }
```

💡 Conceptual Questions:

1. What is F_WRLCK and why do we use it?
2. How does file locking prevent data corruption?

Expected Output

When running the program, create some edge cases and demo how it works. Here is an expected sample logs:

```
Enter file size for Worker 1 (MB): 5
Enter file size for Worker 2 (MB): 50
Enter file size for Worker 3 (MB): 500
Binary files created.
⚠️ Worker exceeded memory limit!
✅ Worker 1 completed.
✅ Worker 2 completed.
```

Assignment Submission:

- Complete all steps, Add all output-screenshot and explanations (if required) to a MS-Word file.
- Add the following declaration at the top of MSWORD file

  /** *

- I declare that this lab is my own work in accordance with Seneca Academic Policy. * No part of this assignment has been copied manually or electronically from any other source* (including web sites) or distributed to other students.* * Name: _____ Student ID: _____ Date: _____
- **********/
- Please submit the Source code (zip all .c, .h, and makeFiles)
- Please answer the following two declarations:
- D1) On a scale from 1 to 5, How much did you use generative AI to complete this assignment?
- where:
- 1 means you did not use generative AI at all
- 2 means you used it very minimally
- 3 means you used it moderately
- 4 means you used it significantly
- 5 means you relied on it almost entirely
- Your answer :
- D2) On a scale from 1 to 5, How confident are you in your understanding of the generative AI support you utilized in this assignment, and in your ability to explain it if questioned?
- where:
- 1 means "Not confident at all – I do not understand the generative AI support I used and cannot explain it."
- 2 means "Slightly confident – I understand a little, but I have many uncertainties."
- 3 means "Moderately confident – I understand the majority of the support, though some parts are unclear."
- 4 means "Very confident – I understand most of the AI support well and can explain it with minor gaps."
- 5 means "Extremely confident – I fully understand the generative AI support I used and can clearly explain or justify it if asked."
- Your answer :

Important Note:

- LATE SUBMISSIONS for labs. There is a deduction of 10% for Late assignment submissions, and after three days it will grade of zero (0).
- This labs should be submitted along with a video-recording which contains a detailed walkthrough of solution. Without recording, the assignment can get a maximum of 1/3 of the total.
- Note: In case you are running out of time to record the video, you can submit the assignment (source